

Performance and QoS in Service-Based Systems

R. William Maule

Information Sciences Department
Naval Postgraduate School
Monterey, CA, USA
rwmaule@nps.navy.mil

William C. Lewis

OPNAV N2/N612
Pentagon 5D150
Arlington, VA, USA
bill.c.lewis@navy.mil

Abstract—We present a systematic method of measuring performance risk for specific implementations of an Enterprise SOA within a Cloud computing framework. The U.S. Department of Defense (DOD) is determining performance baselines for distributed architectures that are internally organized using service interfaces. Such architectures are essential for highly reliable and life critical Cloud applications. We provide baseline performance for an architecture comprised of service components from the service interfaces provided by several major commercial software vendors. Transition from legacy systems into Service-Oriented Architecture (SOA) systems necessitates an infrastructure of federated components. This paper documents a means to measure such an infrastructure. Baselines provide a foundation for assessment of future services including Quality of Service (QoS) contracts and assessment of fused web services within heterogeneous SOA or Cloud architectures.

Keywords—SOA, Performance, Measurement

I. INTRODUCTION

Department of the Navy (DON) is currently operating service oriented architectures in an experimental Cloud Services context. A first step toward a systematic understanding of these experiments is to assemble a test bed that can be used to establish SOA component performance baselines. Understanding is essential to establish reasonable Quality of Service (QoS) metrics. This paper reports that first step. We assemble a test bed of federated SOA components from Commercial off the Shelf (COTS) SOA software, and use that test bed to measure primary components, key variables, and performance metrics.

II. SERVICE INFRASTRUCTURE

Service performance assessment for QoS baselines differs from evaluation of traditional legacy systems. Rather than a two phase process, starting with stand-alone systems and independent software builds, followed by lengthy integration efforts, SOA uses a single phase that continuously introduces (and retires) small, easily integrated web services—each providing specific functions intended for use by any different data consumer that finds it of use. However, the environment of easy reuse and integration seen by data consumers has a high price—the complex software infrastructure that provides that environment. Providing a simple environment, one with versatility to transparently accommodate different networks and components, and to

communicate over the unpredictably varying bandwidths of communications to remote sites, has a high price indeed.

Global organization involving non-DOD networks, as needed to support other nations in disaster relief, requires a mixed federation of networks with different operational objectives and standards. Integration of these infrastructures and their services with the DON is not only difficult organizationally but risks performance and system stability.

Evaluation of stand-alone systems is straight-forward as compared to SOA evaluation. Integration costs have been high but have become predictable. We cannot now reliably estimate cost of federated SOA components or composite applications built on distributed web services in heterogeneous SOA (Cloud computing) infrastructure. This calls into question the promise of lower integration costs and accurate performance prediction.

Composition of services introduces a SOA or Cloud specific propagation error. For example, a composite application may subscribe to a data set from a remote web service, additionally process that data, and then publish the processed data on a generally available web service. A network outage for that web service may cause a default or errors in a composite application that subscribes to it, or for other applications that subscribe to the composite, and on ad infinitum. Fault traces in highly integrated composite applications that use widely dispersed or federated web services can be extremely difficult. Interpreting composite data actually delivered during a failure is often impossible. Unpredictably varying bandwidths and the need to synchronize data over multiple remote sites after an outage complicate an already difficult situation.

The continuing evolution of SOA standards often necessitates inherently risky adaptation of existing services to new standards. Adapting new standards often breaks integration and reuse capabilities, fixing which can be expensive. To prevent this, some minimal set of performance and QoS standards must be developed, agreed to within the federation, adhered to, and modified when necessary. The study described herein is a small step towards such a capability.

Components within typical SOA infrastructure provide the starting point for the evaluation. Specifically, we select and describe major SOA components, and then define metrics. Variables will be drawn from industry best practices and customary usage in commercial SOA software.

III. SERVICE RESEARCH

Service engineering can be broadly defined as a process of developing tools and components, methodologies and algorithms, and applications and systems for the construction of service architecture, components and services [1]. Existing methodology and algorithms do not fully identify performance risk, especially risk from component interaction. Risk is more fully considered in the macro study of which this paper is a subset [2]. The focus herein is on providing a basic structure for component assessment and providing methods for component performance evaluation.

Monitoring, analyzing, and understanding component interactions within an enterprise SOA is difficult yet essential to solving and preventing performance and QoS failures [3]. SOA multiplies the number of independent component interactions that must be monitored, controlled, and debugged by a significant order of magnitude and measurement of QoS is complex due to this increased number of components and processes [4].

Availability of services and resources is a QoS concern as SOA shifts the use paradigm from single to integrated systems [5, 6]. SOA requires measures of individual processes and components, plus the cumulative impact of operations in each layer, e.g., process, service, routing, transformation, etc. [7]. The loosely coupled and heterogeneous nature of SOA necessitates well-defined metrics to diagnose service performance [8].

One technique is to define desirable quality attributes and then trace the metrics required to measure them, and at different levels of abstraction [9]. Another is to assess service granularity and service coupling between services and clients [10]. Measures may include service-oriented process speed, system reliability, throughput, and availability [11]. This paper presents these and additional metrics as baseline measures for SOA QoS assessment, based on field and laboratory experimentation.

Additionally, SOA risk mitigation requires that developers adhere to standards and associated “best practices”. Service functions must be exposed in a standardized way so that they can be re-used across multiple projects [12]. However, this requires uniform application of standard procedures, which may be difficult or impossible in global operations. This problem is addressed by the OASIS SOA Reference Model [13], which describes a process for rapid negotiation of exposure agreements within a federation.

There is also a tradeoff between latency increases for those using fully-featured service implementations [e.g., the “full stack” of SOA components] versus streamlined models with fewer service features. The two approaches provide different latencies for similar services, and apportion system costs differently between service participants and system providers. Optimizing such systems is seldom discussed.

While there are solid heuristics for evaluation of specific service projects [14], heuristics to address the impact of component variations are generally absent. Yet, practical limitation of federation performance risk requires such heuristics. At the service level, some basis for analysis may be adopted from research on architectural frameworks [15].

Performance risk management might apply frameworks for Quality of Service (QoS) assessment in service interaction and user requirements [16, 17]; and service level contracts to formalize component relationships, object interactions, and associated rules. It must be borne in mind that SOA and Cloud systems are constantly changing, cannot be globally harmonized, and that static frameworks must be at a very high level of abstraction, specifying procedures in terms of general but persistent environments rather than for specific but transient environments.

IV. SOA EVALUATION ARCHITECTURE

The SOA Evaluation Architecture (SEA) was established to baseline SOA component interaction. The environment currently consists of two (2) geographically separated physical locations with Internet communication. SEA includes wireless links, and multiple intervening networks. At the two (2) locations are a current total of six (6) physical hosts, supporting thirty (30) virtual servers. SEA includes three (3) different virtualization approaches and multiple SOA configurations.

Industry categorizes SOA components as either “best of breed” or as “suites” (manufacturer integrated components). In addition there are COTS solutions and open source options, with performance tradeoffs for each. If software is “best-of-breed” (e.g., the best ESB) then there may be systems integration costs that will be incurred to make supporting components work together. From an engineering standpoint, the vendor bears systems integration costs of a “suite”, making the initial cost of the suite much higher but potentially lowering implementation and long-term maintenance costs as the vendor can spread these across many customers. From a contracting standpoint, a closed standards “suite” frees the provider from market competition, increasing maintenance costs.

Typically, if an “open source” SOA option is selected then components cost less, but user provided maintenance and integration may cost more than component costs saved, and user provided security is weaker than security integrated and hardened by a “suite” from a commercial vendor. To note is that even commercial vendors of SOA include open source software in their products. Industry standards are built on open source. Often commercial vendors have contributed this code to the open source community. Clearly, evaluating the support and costs of a SOA / Cloud computing system that serves a federation is not simple.

Fig. 1 models the performance assessment tiers for homogeneous single-node SOA. Complexity increases significantly when interactions between heterogeneous components and multiple nodes must be considered. Such discussion is beyond the scope of this paper. Note also that only some of the tiers and variables are addressed in this paper. Future research on additional variables is planned. Readers familiar with service models from the major vendors will have noted that Fig. 1 models are only conceptual. Each vendor’s implementation is different and greatly elaborates its conceptual model.

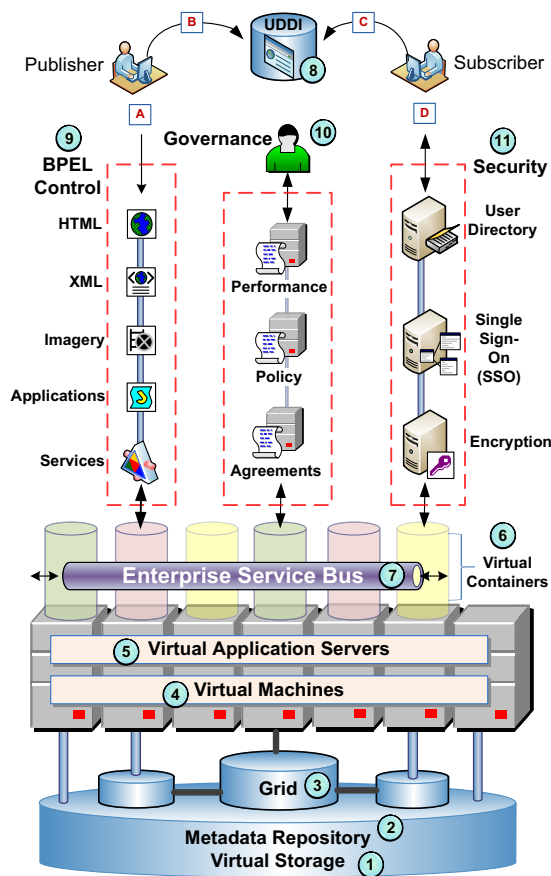


Figure 1. SEA performance assessment framework.

The center performance assessment tier in Fig. 1 shows the complexity of one very simple implementation. Fig. 1 is intended to help convey a rough idea of what different software and hardware components do within SOA and thus provide a foundation for understanding the performance variables identified in Table I.

TABLE I. SEA PERFORMANCE ASSESSMENT VARIABLES.

#	Component	Component Functionality Categories
1	Storage	Virtualization performance and virtual system interfaces issues
2	Repository	Database performance, cluster issues, virtualization options
3	Grid	Database and application management, process and agent performance options
4	Virtualization	Virtual machine management, systems management, performance options
5	Servers	Java Virtual Machine performance, J2EE and .NET configuration options
6	Containers	Interoperability issues, deployment processes and performance, QoS
7	Service Bus	QoS, interoperability, throughput, latency; deployment options
8	Registry	QoS, object permissions, throughput, latency, service performance
9	Orchestrate	Performance of process monitors and procedure calls; latency, throughput
10	Governance	Service agreements, policy enactment; latency, throughput
11	Security	Authentication and authorization latency and throughput, encryption

There are many performance variables in the storage tier, dependent on the intended use of the services, ranging from small and optimized for specific local functions at one extreme, to large and capable of federation wide backbone and archival operations at the other extreme. There are performance variables associated with media selection (iSCSI, wireless, etc.), and associated metrics for throughput, latency and capacity.

The metadata repository, which is functionally a database or XML schema, inherits the performance characteristics of a traditional database. Clustering and virtualization impact the repository. Traditional metrics for database analysis are applied to the metadata repository. Plus, performance metrics specific to user access, virtualization and clusters.

Table II provides the results of a test of basic repository functionality in the SEA environment and exposes some basic metrics. Eighteen (18) tests were run with only one (1) failure and no errors for an overall success rate of 94.44%. Time required to run the assessment tests was 0.437 seconds. Tests captured typical user sessions in secured operations and then replayed those interactions with increasing load and number of users. For QoS capability validation, future tests might record Table II metrics at periodic time intervals while increasing load on the Repository, and compare recorded metrics with baseline measures of Table II.

TABLE II. SEA METADATA REPOSITORY TEST METRICS.

Metric	Status	Time
Test for Wrong Directory Structure	Success	0.016
Test Properties File For False Positive	Success	0.000
Test Properties File For CMEE	Success	0.000
Test Properties File For Database	Success	0.000
Test Eventing Setup	Success	0.000
Test Encrypt Password	Success	0.000
Test SSL URL	Success	0.281
Test Single Substitution	Success	0.063
Test Single Match	Success	0.000
Test Number Sub String Match	Success	0.000
Test Reg Ex Match	Success	0.000
Test XQL Tool Loop Children	Success	0.000
Test UTF8 Encoded XML	Success	0.015
Test Reading Build Tag From Jar	Success	0.000
Test Invalid XML String	Success	0.016
Test Parse Roundtrip	Success	0.000
Test XPath Query	Success	0.000

Current commercial service offerings in grid computing tend to focus on either the database and storage tiers or else on the application and process tiers. Grid performance measurement is typically taken by an agent added to each node's host. That agent remains resident on the host and gathers information from specified processes. Use of an agent can add to host latency, reduce the host's working memory, and thus detrimentally impact other processes. Multiple agents on a host may conflict. Conflict will further degrade performance. Host introduction risk must be minimized by design rather than simply assumed to be zero. The ability of a Grid agent not only to monitor a process but to intervene and control that process increases the need for Grid agent design to minimize risk.

Virtualization is assumed throughout the SOA and as the basis for the host cloud, but in Fig. 1 is presented as Number 4 at the level of the physical host. These are the traditional virtualization technologies—SEA tests included VMware, Xen and Hyper-V. In the SEA test environment, each of these virtualization approaches has shown its strengths and weaknesses—discussion of which is beyond this paper. Virtualization performance metrics address virtual machine creation, cloning, failover, distributed deployments, and template usage.

The application server is the middle tier host for services and applications. The application server and its virtual machine instances comprise what is typically referred to as “middleware” (Table III) and are mutually supporting. In operation, if the database tier fails then data is not available, not accurate, or users cannot authenticate and login to the services. If the middleware tiers experience performance problems then services start to fail.

TABLE III. SEA MIDDLE TIER VARIABLES.

Variables	Description
Virtual Server	Instance that runs in its own Java Virtual Machine (JVM) and has its own configuration.
Cluster	Multiple virtual servers run simultaneously on one physical server to increase performance and scalability.
Virtual Hosts	Set of host names to which server instances respond. Host names map to the IP address.
Machine	Virtual representation of a physical computer that hosts one or more server instances (servers).
Work Managers	Set of request classes and thread constraints: J2EE modules, EJBs, RMI provide metrics.
Startup / Shutdown	Java classes/programs to provide custom, system-wide services for applications.

Services tested here were run on virtual servers in the SEA environment using Java 2 Enterprise Edition (J2EE). In addition to basic server functionality, Table III performance variables enable assessment of the Java Virtual Machine(s) (JVM) and containers and also end-to-end SOA throughput. Performance variables for garbage collection, process kills and restarts, latency within or across processes, and other important functionality, are not yet included.

Java “containers” exist within the application server. They interface directly with hardware, and implementations of user visible virtual applications or services in Java are deployed to the server/container. Latency imposed by any Java components or processes will slow down all services.

Containers are the building blocks of service deployment. Container metrics are for the processes they support. SEA lab and Navy field tests have evaluated containers supporting a range of configurations, from a simple container in one hardware box (a JVM and application server) at one extreme, to a container that spanned geographically distributed hosts at the other. So,

there are options for container deployment and there are different QoS metrics for each.

The ESB implementation is a potential choke point. Failure of service interoperability or message throughput (latency) over an ESB constitutes a federation-wide system failure. Future research will consider additional interoperability measures for logical correctness, latency, transformation exactness, and service interoperability.

Current commercial techniques tend to focus on ESB operational variables, or on content functions. Performance variables in content-based routing, and associated metrics, could include:

- *Service mediation*—Metrics on request and response messages and routing between service endpoints
- *Conditional messages*—Metrics on processing and transformation between service endpoints
- *Dynamic message routing*—Metrics on message content, multicast or multi-path messages; dynamic service provisioning, versioning or transformation
- *Conditional checks*—Metrics on branch statements, to retrieve the value of a data element or multiple data elements that determine routing logic
- *Multiple destination services*—Metrics on data element values; transformations applied to messages going to multiple destinations, SLA-based changes
- *Service Level Agreements (SLAs)*—Contractual thresholds for availability, performance, and queuing.

Table IV proposes SEA metrics for ESB tests, to be gathered at each node. Baseline metrics would be established over a range of loads for each node, then the nodes would be synchronized within their local network. Then the network would be loaded, and finally concurrent applications would be run.

TABLE IV. ESB METRICS FOR ASSESSMENT BASELINES.

Metric	Description
Average Execution Time	Time between message reception at the transport and exception or response.
Total Number of Messages	If the transaction aborts, and message is placed back in the queue, each retry de-queue counts as a separate message.
Messages With Errors	Number of messages that resulted in an exit with the system error handler or in an exit with a reply failure action.
Success / Failure Ratio	(Total Messages - Number of Messages with Errors) / Messages with Errors
Security	Messages with WS-Security errors.
Validation	Count of validation actions that have failed. Metric applies to proxy services.

Registry metrics assess service invocation, metadata management processes, transport, and QoS. Enforcement mechanisms report whether component registrations meet contractual obligations. Metrics can be applied for each

step of the registration process, including the Web Services Description Language (WSDL) forms that have been published to the Universal Description Discovery and Integration (UDDI) service, reference information for the service providers, the endpoints and interfaces through which programs can connect to these services, policies associated with the Registry or an individual service, and XML style sheet translations.

Orchestration environments support coordination and management of services, typically providing governance interfaces such as Business Process Management (BPM) and tools to build and control processes and web service interactions, such as Business Process Execution Language (BPEL). Metrics are needed for BPEL interface functionality, performance of the Governance system, web service policy enforcement, and finally for BPEL throughput and latency to include remote procedure calls.

Governance systems support SOA management over longer time intervals, and typically log service metrics for later use. Agents can help provide service metrics. Governance supports administrative actions to enforce policies, service agreements, and runtime BPEL procedures. BPEL flow metrics proposed include service levels on BPEL processes and on dependent partner links.

Grid controls can record key metrics for web services, distinguishing actual requests from synthetic transactions. Governance metrics could include J2EE container monitors, JVM representations, Java Messaging Service (JMS) linkages, and connectors to Business Activity Monitoring (BAM) systems. Potentially, output from Governance software can appear on BAM dashboards, in which case dashboard variables would assess compatibility with analytic and decision support software. Some specific variables and metrics to be evaluated would include:

Repository and Registry Variables – These processes give visibility into assets and relationships:

- *Collection*—automatically populate both the Repository and Registry with existing services and support new asset creation by synchronization with developer tools
- *Categorization*—organize and advertise services and procedures
- *Management*—Discover, map and remember new dependencies, and also record metrics on service impact.

Lifecycle Variables – Automation of repeatable processes for registration, life cycle management, with metrics for:

- *Workflows*—Based on standards and best practices
- *Policies*—Trust in service quality, including QoS, performance, and security
- *Standards*—Dissemination of architectural standards and tracking of architectural compliance.

Analytic Variables –Track and report contract metrics during the service lifecycle:

- *Reuse*—lifespan and number of consumers per service
- *Compliance*—Contract compliance, adjudications of disputes
- *Return-on-Investment*—for all service participants.

Web Services Policy (WS-Policy), and specifications WS-PolicyAssertions and WS-PolicyAttachment, can define conditions under which a service is to be provided and provide metrics based on performance of the service [18]. WS-Policy metrics concern data processing by single and composite services. Service Level Agreements (SLAs) define what service consumers have agreed to accept. QoS metrics are taken from all service participants to tell a global story. Rules can trigger alerts when SLA QoS specifications are not met. Metrics for SLA may include: success rate (success ratio / failure ratio), message count, error count, failover/retry count, validation error count, WSS error count, response time (minimum response / maximum response).

Security for SOA is mission critical but beyond the scope of this paper. Aspects of security tested in the SEA environment have been previously published [19].

V. SEA PERFORMANCE TESTS

We ran SEA tests against both components and integrated suites of components to simulate realistic ship, shore, and distributed operations. The objective of the tests was to establish *preliminary* baseline variables, and show that it is possible to understand service performance under stress in federated SOA and Cloud Computing systems.

Test Scenario #1 (Fig. 2) simulated services within the hull of a ship, as would be expected when communications are down or disrupted. Test Scenario #2 simulated services within a battle group, the CVN serving as the network operations center (NOC) or “hub”. The hub’s “spokes” were the other ships in the group. Test Scenario #3 simulated CVN to shore NOC communications, including both steady state and post-interruption transients. Test Scenario #3 and Test Scenario #4 simulated a link from a CVN to a littoral vessel that then coordinated with a mobile medical unit ashore. #3 and #4 imposed extreme communication conditions to assess distributed SOA in difficult but expected environments, such as a hurricane. Due to space limitations, only results from a subset of the first scenario are presented.

Load tests used recorded SOA processes captured from simulated users in emulated operations—which were then compiled into scripts. Agents were programmed to activate the scripts in different sequences, the agents thereby acting as virtual users. The networks were live and fully operational, as were the SOA components and services. Agents were typically provisioned in groups of ten (10), with their actions layered such that processes increased as the number of users increased.

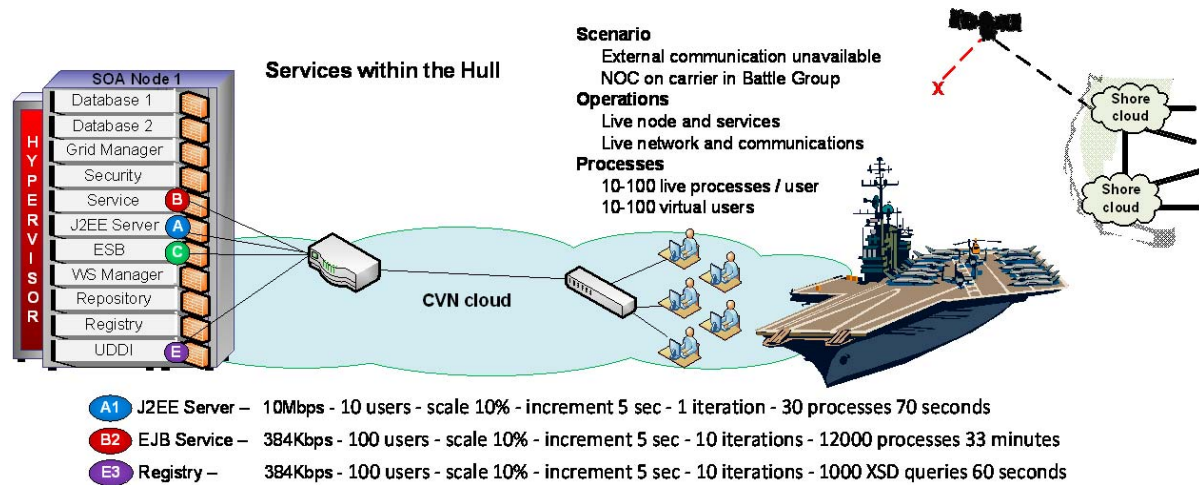


Figure 2. Test scenario #1 for services within the hull.

For example, in several tests 10 users were added every 5 seconds, with each agent (virtual user) running the same test multiple times (multiple iterations). In this manner the number of processes increased exponentially as the number of users increased linearly, to form an upward curve toward peak use, and a down sloping curve toward completion. In this manner the stress test assumed large usage suddenly, rather than even use over time, to emulate expected load escalation during a natural disaster relief effort.

Operational network impairment devices placed between the agents and components were designed to simulate network communications, with low-bandwidth links for challenged units, typical satellite bandwidth for CVN to shore communications, and broadband for services within CVN hull. Signals were collapsed or throttled to emulate channel prioritization.

The first performance baseline, Test Scenario #1, charted application server performance within the CVN hull. J2EE Application Server tests measured effect of ten (10) users on a 10 Mbps communication link (LAN). A Java client was used, with no errors. Tables V and VI present test results. The total number of transactions was 19, with 57 hits, and 1423 KB transferred. One (1) minute was required to complete 19 transactions with a cumulative 57 processes. All transactions were successful, with an average 0.1 standard deviation for each transaction from the average.

Next, we wanted to examine the internal workings of the server under load. The Enterprise Java Bean (EJB) service tests assessed the impact of request processing within the application server container, which is where the bulk of the processing for a SOA typically occurs. The test was for 100 users on a 384 Kbps communication link (to emulate a saturated LAN). The number of users began with 10. The number of users was then incremented by 10% every 5 seconds until reaching the peak of 100 users.

Additionally, each user initiated the processes 10 times (10 iterations), with the start of a process beginning 1 second after completion of the previous process.

TABLE V. APPLICATION SERVER TRANSACTIONS.

<i>J2EE Server</i>				
Name	Value	Min	Max	Avg
Active Virtual Users	10	0	10	4.833
Virtual Users with Errors	0	0	0	0
Transactions Per Second	0.404	0.014	0.459	0.253
Pages Per Second	1.2	0.133	1.4	0.76
Hits Per Second	1.2	0.133	1.4	0.76
Kilobytes Per Second	36.568	2.005	36.568	18.971
Totals				
Transactions	19			
Transactions with Errors	0			
Pages	57			
Hits	57			
Kilobytes	1423			

- Average .253 transactions per sec, max .459, min .014
- Average .76 hits per second, maximum 1.4, minimum .133
- Average 18.971 KB per second, max 36.568, min 2.005

TABLE VI. APPLICATION SERVER PERFORMANCE.

<i>Performance</i>							
Name	Min	Max	Avg	Pass	Fail	StdDev	90th%
Cumulative	16.197	16.873	16.426	19	0	0.304	16.873
[1] Server	0.024	0.36	0.116	19	0	0.163	0.36
[2] Service	0.179	0.421	0.245	19	0	0.118	0.421
[3] Application	0.577	0.812	0.685	19	0	0.097	0.812

A Java client, with no errors, recorded:

- Average .517 transactions per sec, max .763, min .024
- Average 8.787 hits per second, max 15.733, min .6
- Average 45.249 KB per second, max 108.493, min 5.96

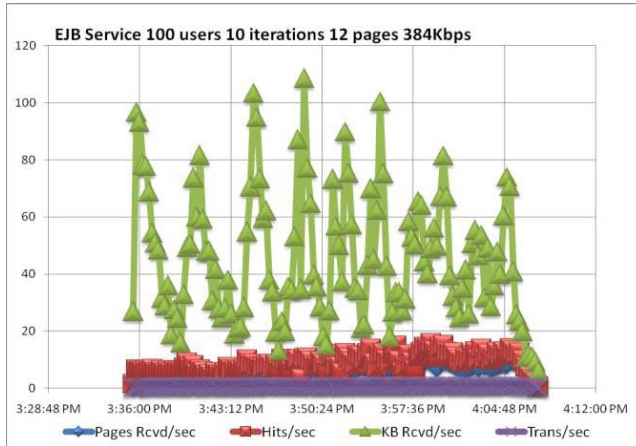


Figure 3. Service use-case test #1.

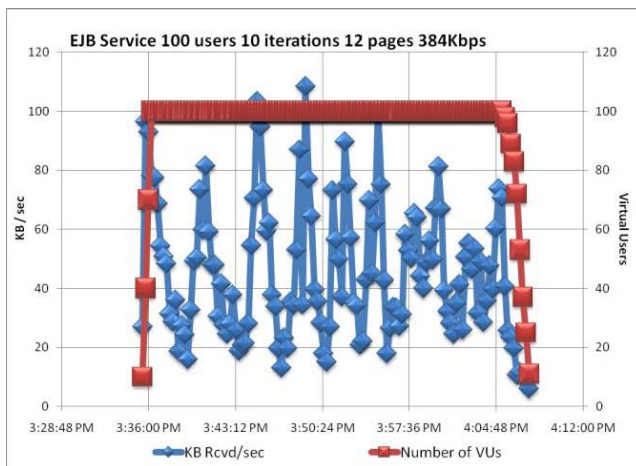


Figure 4. Service use-case test #2.

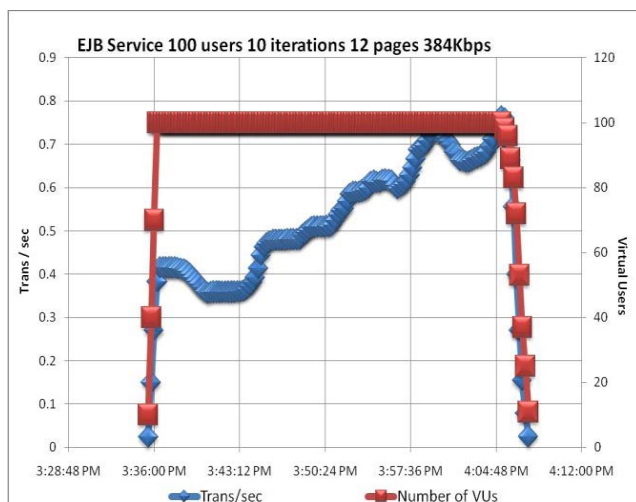


Figure 5. Service use-case test #3.

The total number of transactions was 1000, for which there were 17,003 hits, and 87,556 kilobytes transferred. Thirty-two (32) minutes were required to complete 1000 transactions; each transaction required 12 steps. All

transactions were successful, with an average 5.45 standard deviation for each transaction from the average. Figures 3-5 illustrate the performance baselines for EJB processes within the communication context. To note is the length of time required to complete processes. This would be due to the requirement of those processes to generate data sets at each step and to render that data to screen.

The charts illustrate the impact of the processing load at each step, incremented exponentially over time, compounded for both users and number of processes. The increase in the efficiency of transaction processing, with load over time, may indicate some manner of internal caching or other efficiency measure, or simply that processing iterations had completed for a given set of users. Additional tests would be necessary to determine cause and effect.

VI. CONCLUSION

Fitting commercial enterprise SOA packages to a specific application, even with supporting measurement capabilities, is highly complex and expensive. We have demonstrated that a program of systematic testing can compile a metric profile of the fit for four simulated Department of the Navy task forces. We have applied the metric profile to assess the SOA/Cloud networks ability to meet mission requirements.

The metrics gathered above are likely to be useful in computing performance assessment and QoS assessment of primary SOA/Cloud computing components.

The examples presented constitute a preliminary model of assessment techniques. Represented components, and the small subset of data presented in this paper, are intended to illustrate the technical constructs and supporting variables that can be used as a framework for evaluation of network fit to its intended application.

The tables of variables and metrics and the sample rendering of test data sets presented above, are not intended to provide a definitive solution. Rather, they demonstrate the existence of a method. We have demonstrated assessment of performance and QoS in distributed enterprise SOA, and done so in four communication-challenged environments for which the Enterprise SOA/Cloud computing packages we assessed were not designed. We hope to open a dialog on performance and QoS measurement techniques.

ACKNOWLEDGMENT

The author wishes to acknowledge OPNAV N61 and N81 for their support for this research.

REFERENCES

- [1] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and Design*, Prentice Hall, New York, 2005.
- [2] Maule, R., and Lewis, B. *SOA Baseline Architecture Assessment Project: Report 4: Summary Findings and Recommendations*, Washington, DC: Department of Navy, OPNAV, N2/N6, 2009.

- [3] T. Parsons, A. Mos, M. Trofin, T. Gschwind, and J. Murphy, "Extracting Interactions in Component-Based Systems," *IEEE Transactions on Software Engineering*, Vol. 34, No. 6, 2008, pp. 783-799.
- [4] Liu, G., Zhu, Z., Li, Y., Li, D., and Cui, J. "A New Web Service Model Based on QoS", *International Symposium on Intelligent Ubiquitous Computing and Education*, 2009, pp. 395-399.
- [5] Luo, J., Li, Y., Pershing, J.; Xie, L., and Chen, Y. "A Methodology for Analyzing Availability Weak Points in SOA Deployment Frameworks", *IEEE Transactions on Network and Service Management*, Vol. 6, No. 1, 2009, pp. 31-44.
- [6] Rajan, H., and Hosamani, M. "Tisa: Toward Trustworthy Services in a Service-Oriented Architecture", *IEEE Transactions on Services Computing*, Vol. 1, No. 4, 2008, pp. 201-213.
- [7] Lee, Y. "Event-driven SOA Test Framework Based on BPA-Simulation," *First International Conference on Networked Digital Technologies*, 2009, pp. 189-194.
- [8] Her, J., Choi, S., Oh, S., and Kim, S. "A Framework for Measuring Performance in Service-Oriented Architecture", *Third International Conference on Next Generation Web Services Practices (NWeSP 2007)*, 2007, pp. 55-60.
- [9] Shim, B., Choue, S., Kim, S., and Park, S. "A Design Quality Model for Service-Oriented Architecture", *15th Asia-Pacific Software Engineering Conference (APSEC 2008)*, 2008, pp. 403-410.
- [10] Xiao-jun, W. "Metrics for Evaluating Coupling and Service Granularity in Service Oriented Architecture", *International Conference on Information Engineering and Computer Science (ICIECS 2009)*, 2009, pp. 1-4.
- [11] Gao, J., Wu, Y., Chang, L., and Meldal, S. "Measuring Component-Based Systems using a Systematic Approach and Environment", *Second IEEE International Workshop on Service-Oriented System Engineering (SOSE 2006)*, 2006, pp. 121-129.
- [12] G. P. Kumari, B. Kandan, and A.K. Mishra, "Experience Sharing on SOA Based Heterogeneous Systems Integration," *2008 IEEE Congress on Services*, Honolulu, HI, 7-11 July 2008.
- [13] OASIS. Reference Model for Service Oriented Architecture 1.0, 12 October, 2006. URL: <http://docs.oasis-open.org/soa-rm/v1.0/soa-rm.pdf>
- [14] T. Hau, N. Ebert, A. Hochstein, and W. Brenner, "Where to Start with SOA: Criteria for Selecting SOA Projects," *Proceedings of the 41st Hawaii International Conference on System Sciences*, Waikoloa, HI, 7-10 January 2008.
- [15] T. Roach, G. Low, J. D'Ambra, "CAPSICUM – A Conceptual Model for Service Oriented Architecture," *2008 IEEE Congress on Services*, Honolulu, HI, 7-11 July 2008.
- [16] S. W. Choi, J. S. Her, and S. D. Kim, "Modeling QoS Attributes and Metrics for Evaluating Services in SOA Considering Consumers' Perspective as the First Class Requirement," *IEEE 2nd Asia-Pacific Services Computing Conference*, Tsubuka Science City, Japan, 11-14 December 2007.
- [17] R. Maule, "Quality of Service Assessment in SOA Synchronous Networked Communications," *Proceedings of the 2007 International Conference on Computing, Communications and Control Technologies (CCCT 2007)*, Orlando, FL, 12-15 July 2007.
- [18] <http://en.wikipedia.org/wiki/WS->*
- [19] Maule, R., and Lewis, W. "Security for Distributed SOA at the Tactical Edge", *Proceedings of the IEEE 2010 Military Communications Conference (MILCOM 2010)*, 31 October–3 November, San Jose, CA, 2010, pp. 1622-1627.